

TaxDesk Codebase Mastery, Owning the Actual Code

The deep dive teaches the system's ideas. This document teaches the repository itself, file by file, function by function, down to single lines where it matters. The final test it aims at, if the original developer vanished, could you continue building from this repo alone.

Read with the code open. Every claim here was checked against the source on 2026-08-13, with 18 tests passing.

Priorities, so you spend attention where it pays.

- Tier 1, must understand deeply: queries.py, generation.py, periods.py, deps.py, migrate.py
- Tier 2, should understand: dashboard.py, clients.py, onboarding.py, templates, tests, seed.py
- Tier 3, read once, move on: **init** files, pyproject.toml, base.html

1. The Map

```
taxdesk-ai/
├─ pyproject.toml      identity, pinned dependencies, pytest config
├─ taxdesk.db          the database, created at runtime, never in git
├─ app/
│   ├─ main.py         assembles the app, migrations at startup
│   ├─ deps.py         template engine + per request db connection
│   └─ routes/
│       ├─ onboarding.py root folder, discovery, confirm clients
│       ├─ clients.py   home redirect, client list/detail, services
│       ├─ periods.py   months, generation, task statuses, closing
│       └─ dashboard.py dashboard and priority, read only
│   └─ services/
│       └─ generation.py financial year, due dates, task generation
│   └─ db/
│       ├─ migrate.py   connect() and the migration runner
│       ├─ queries.py   every SQL statement the app runs
│       ├─ seed.py      fake data for development, run by hand
│       └─ migrations/
│           └─ 001_initial_schema.sql
│   └─ templates/      base + one html file per page
└─ tests/
    ├─ conftest.py     fresh migrated temp database per test
    ├─ test_generation.py
    ├─ test_task_status.py
    └─ test_dashboard.py
```

File	Responsibility	Called by	Calls
main.py	assemble app, startup migrations	uvicorn	migrate.py, deps, all routes
deps.py	connection lifecycle, templates	FastAPI per request	migrate.connect
onboarding.py	folders become clients	FastAPI	queries, pathlib
clients.py	client pages, service ticks	FastAPI	queries
periods.py	months and task statuses	FastAPI	queries, generation
dashboard.py	glance pages	FastAPI	queries
generation.py	generation logic	periods.py, tests	sqlite via conn
migrate.py	open db, apply migrations	everyone	sqlite3
queries.py	all SQL	every route, tests	sqlite via conn
seed.py	dev data	a human	migrate.py

Documentation versus code, mismatches found

The prompt rule is to report these, not hide them.

1. README's "Next Milestone" section still says do not build UI first and points at the data model. That milestone finished weeks ago, v0.5 shipped a full UI. The README needs a refresh pass.
2. folder-structure.md's draft layout includes app/static/. No static folder exists yet, styling is inline in templates. Harmless, but the draft and reality differ.
3. app/db/seed.py contains its own generate_tasks(), nearly identical to the one in app/services/generation.py. Not a doc mismatch, a code duplication, explained in the review section.

2. The Runtime, What Actually Happens When You Type the Command

```
venv/bin/uvicorn app.main:app --reload
```

- venv/bin/uvicorn, a Python program installed inside the project's virtual environment, an isolated folder of installed packages so this project's libraries cannot clash with another project's
- the operating system starts it as a process, one running program with its own memory
- uvicorn reads `app.main:app`, meaning import the module `app.main` and inside it find the variable named `app`
- importing `app/main.py` runs its top level lines. The import statements pull in the route modules, and importing `THOSE` runs their decorators, which is how every route gets registered before anything serves
- `lifespan()` runs, opens the database (creating the file if absent), applies pending migrations, closes

- uvicorn binds port 8000, meaning it asks the operating system to hand it every connection arriving at that number, and waits

A port is just a numbered mailbox on the machine. localhost:8000 means the mailbox numbered 8000 on this same computer.

3. Framework Magic, Opened Up

Nothing the framework does is magic, it is registration plus lookup.

The decorator

```
@router.get("/dashboard", response_class=HTMLResponse)
def dashboard(...):
```

A decorator is a function that receives the function below it. `router.get("/dashboard")` builds a registrar, and applying it stores an entry in the router's table, GET plus /dashboard maps to this function object. This happens once, at import time. At request time FastAPI just looks up the table. Delete the decorator and the function still exists, but no URL reaches it.

Depends(get_db)

```
def dashboard(request: Request, conn: Connection = Depends(get_db)):
```

At import time, FastAPI reads the signature and notes, this route needs the result of `get_db`. At request time, before calling the route, it calls `get_db()` in `app/deps.py`. `get_db` is a generator, it runs until `yield conn`, and the yielded connection is what lands in the route's `conn` parameter. When the route finishes, FastAPI resumes the generator, the code after `yield` runs, commit on success, close always. So the connection's whole life brackets the route call, and no route can forget cleanup because no route performs it.

await request.form()

The form body arrives over the network in pieces. `await` pauses this one request's function at that line, hands the thread back to the event loop so other requests can progress, and resumes when the body is complete. That is also why the three form reading routes are `async def` while every other route is a plain `def` running on FastAPI's thread pool.

Path parameters

/clients/{client_id} plus a parameter declared `client_id: int` makes FastAPI extract the path segment and convert it to int before your code runs. A non number gets rejected by the framework, your function never sees it.

4. Tier 1 Files

app/db/queries.py

Why it exists. One boundary owning ALL SQL. Separation solves two problems, routes stay readable, and any query is findable and changeable in one place. Its most important property, Dashboard and Priority share the same functions, so their numbers cannot drift.

Dependency map.

```
queries.py
├─ called by → every route module, tests
└─ calls      → nothing of ours, only the connection it is handed
```

The full code walkthrough your prompt asked for, on the smallest function, every token.

```
def get_client(conn: Connection, client_id: int) -> Row | None:
    return conn.execute(
        "SELECT id, name, folder_path, phone, email, notes FROM clients WHERE id =
?",
        (client_id,),
    ).fetchone()
```

- `def` begins a function, a named block that receives inputs and returns an output
- `conn: Connection`, the open database handle. It was created in `deps.get_db()` by `migrate.connect()` and passed down. Passing it in, instead of creating it here, means tests can pass a temp database and the function never controls its own lifecycle
- `client_id: int`, born in the URL. The browser asked for `/clients/7`, FastAPI extracted 7, converted it to int, the route passed it here untouched
- `-> Row | None`, the contract, either one row or None, and the caller must handle both
- `conn.execute(sql, params)` sends the SQL text and the parameters separately to the SQLite engine. Separately is the security, the `?` is a placeholder the engine fills AFTER parsing, so the value can never be mistaken for SQL. Gluing the value into the string instead is the SQL injection hole
- the SQL, `SELECT` names the columns wanted, `FROM` names the table, `WHERE` keeps only rows passing the condition. `id` is the primary key, so at most one row can pass
- `(client_id,)`, a one element tuple, the comma makes it a tuple, one value per `?` in order
- `.fetchone()` asks for the first result row, or None when there is none. The route turns that None into a 404

The journey the prompt asked to see.

```
browser GET /clients/7
→ FastAPI extracts 7 → client_detail(client_id=7)
→ get_client(conn, 7) → SQLite finds the row by primary key
→ Row comes back → template renders client['name']
```

Everything else in queries.py is this same shape at different sizes. The two worth extra attention, pending_counts() with its GROUP BY, and pending_tasks() which builds its WHERE incrementally, appending AND t.service_type = ? and the LIKE clause only when filters are present, params list growing in step with the SQL string, always through placeholders.

app/services/generation.py

Why it exists. Generation is the product's core rule, and it lives where tests can import it without any web machinery. Delete this separation and the logic moves into a route, still runs, but every test of it then needs a fake HTTP request, and reuse from a future CLI or scheduler becomes copy paste.

The function that IS the product, at three levels.

```
cursor = conn.execute(
    "INSERT OR IGNORE INTO compliance_tasks (client_id, period_id, service_type)"
    " SELECT client_id, ?, service_type FROM client_services WHERE active = 1",
    (period_id,),
)
created = cursor.rowcount
```

Level 1, beginner. Copy every active client service pair into the tasks table for this month, skip pairs already present, count what was new.

Level 2, programmer. INSERT ... SELECT is one statement that reads and writes inside the engine, no Python loop, no row ever crosses into Python. OR IGNORE converts a UNIQUE violation from an error into a silent skip. cursor.rowcount counts only genuine inserts, which is how the page can say 13 new, then 0 new.

Level 3, engineering. Idempotency lives in the database constraint, not in Python checks. A double click, a retry, a bug that calls this twice, all land on the same UNIQUE index and bounce. The code does not have to be careful because the schema is.

Also here, financial_year(), pure arithmetic on month and year, and DUE_DAY_RULES, the placeholder dictionary that stays all None until dad's answers arrive, with due date backfill already wired and tested via monkeypatch.

app/routes/periods.py

Why it exists. The Excel replacement pages, months and their tick lists. It owns every WRITE route in the tasks area, and therefore owns enforcing the closed period rule.

Dependency map.

```
periods.py
├─ depends on → deps.py (get_db, templates)
├─ calls      → queries.py (periods, tasks)
├─ calls      → generation.py (financial_year, generate_tasks)
└─ renders    → periods.html, period_detail.html
```

The business rule worth locating precisely. A closed month refuses writes. It is enforced twice, in `generate()` before generating and in `set_task_status()` before flipping a status, both by loading the period row and checking `period["status"] == "closed"`, answering with a 303 redirect carrying `error=closed`. The rule lives in the route layer today because it gates web actions. If a scheduler ever generates tasks, the check must move down into `generation.py` so every caller passes through it, a known relocation, not a surprise.

Value trace for a status change, the full journey of one string.

```
"done"
  from the hidden input in period_detail.html
  → POST /tasks/42/status form body
  → await request.form(), form.get("status")
  → whitelist check, status in ALLOWED_STATUSES, reject anything else
  → queries.set_task_status(conn, 42, "done")
  → UPDATE sets status, completed_at, completed_source='manual'
  → commit in get_db teardown → 303 → the page rereads the truth
```

app/deps.py

Why it exists. Two shared resources, the template engine and the connection lifecycle, needed by all four route modules. Without it, either every route module builds its own (drift), or routes import from each other (coupling).

The lifecycle, precisely, because this is where resources live and die.

```
request arrives
  → FastAPI calls get_db()
  → connect(db_path()) opens taxdesk.db, PRAGMA foreign_keys ON
  → yield hands the connection to the route
  → route runs, maybe raises
  → generator resumes, success path commits, finally closes
  → connection destroyed, nothing leaks, one per request, never shared
```

`db_path()` reads the `TAXDESK_DB` environment variable so tests and scratch runs can aim the entire app at another file. Defined by the shell that starts the process, read at each call, nothing cached.

app/db/migrate.py

Why it exists. Structure must reach every machine identically while data stays local. `connect()` is also the codebase's single door to SQLite, holding the two settings everything depends on, `PRAGMA foreign_keys = ON` per connection, and `check_same_thread=False` with `row_factory` for name based access.

The runner's whole algorithm is three lines, worth reading in the file, already applied set, sorted files, list difference. Then per pending file, `executescript`, insert into the logbook, commit, one atomic unit per migration. Delete the logbook insert and every restart would try to recreate existing tables and crash.

5. Tier 2 Files, Shorter

app/routes/dashboard.py

Read only, both pages. `pick_period()` resolves which month, URL wins when valid, else `default_period()`. `dashboard()` calls `pending_counts` and `pending_clients`. `priority()` calls `pending_tasks` with optional service and search, and validates the service against `SERVICE_TYPES`, an unknown value silently becomes no filter.

app/routes/clients.py

`home()` is the traffic cop for `/`, onboarding when no root folder, else dashboard. `client_detail()` carries its own copy of the period from URL logic, see the review section. `save_services()` reads the checkbox list and calls `set_service` per service type, unticked means active 0, never DELETE.

app/routes/onboarding.py

The trust boundary file. `confirm_clients()` recomputes real subfolders and only accepts submitted names inside that set, which is why a crafted `../evil` dies silently. `folder_candidates()` is the shared discovery helper.

app/templates/

Templates receive a dict and display it, they never call our code. The one behavior to know, `{{ value }}` escapes HTML on the way in. Wide tables and forms live in `period_detail.html`, the busiest one.

tests/, and how to read code THROUGH tests

Each test is executable documentation. The table your prompt asked for, what breaks if each is deleted.

Test	Production code it exercises	Bug it would catch
<code>test_second_generation_run_creates_nothing</code>	<code>generation.generate_tasks</code> , the UNIQUE index	duplicate protection silently broken

Test	Production code it exercises	Bug it would catch
test_dashboard_and_priority_always_agree	pending_counts + pending_tasks	the two pages drifting apart
test_back_to_pending_clears_completion_trace	queries.set_task_status	stale completion data lying
test_due_dates_fill_once_a_rule_lands	DUE_DAY_RULES backfill	dad's future answers not taking effect
test_default_period_prefers_open	queries.default_period	dashboard opening on a closed old month

The trace behind every one of them is identical, fixture builds a temp database via migrate(), test calls production functions directly, asserts on rows. No HTTP anywhere, deliberately, tests lock logic, the manual click through checks wiring.

app/db/seed.py

Six fake clients, July 2026, generated tasks, two interesting statuses, prints the counts. Run by hand only. Contains the duplication flagged below.

6. Configuration, All of It

value	defined	used by	if it changes
TAXDESK_DB	shell env var	deps.db_path per call	whole app targets
another db file			
DEFAULT_DB_PATH	migrate.py	deps, scripts	db file moves,
gitignore must match			
MIGRATIONS_DIR	migrate.py	the runner	migrations live
elsewhere			
DUE_DAY_RULES	generation.py	generate_tasks, due_date_for	due dates start
filling on next generate			
SERVICE_TYPES	queries.py	routes, templates, generation display	a fifth
service appears everywhere EXCEPT		the schema CHECKs, see review	
pins	pyproject.toml	humans installing	dependency versions
drift if ignored			

No secrets exist in this project, nothing to manage yet.

7. The Dependency Graph, and Its Health

```
main.py → routes → deps.py → migrate.py
      ↓
      queries.py, generation.py → (the connection) → SQLite
      ↓
      templates (render only)
```

Verified by grep on 2026-08-13, all imports point downward, no cycles. Coupling is low, route modules never import each other.

The one module to watch is queries.py, twenty two functions and growing by milestone. Fine today, and the natural split later is by domain, queries/clients.py, queries/tasks.py, a mechanical refactor when the file stops fitting in your head.

8. Where Do I Make This Change

Add a field to Client, say gstin

1. new file app/db/migrations/002_add_client_gstin.sql with ALTER TABLE clients ADD COLUMN gstin TEXT, the runner picks it up on next start
2. queries.py, add the column to get_client's SELECT and to whatever write function edits clients when editing exists
3. client_detail.html, display it
4. a test asserting it survives a round trip Nothing else. Routes pass rows through untouched, templates display what they get.

Add a new page

1. new function in the fitting route module, or a new module plus one include_router line in main.py
2. any new SQL goes in queries.py, never inline
3. a template extending base.html, a nav link if permanent
4. tests for whatever logic it adds

Change a business rule, example, closed months may still generate

The rule lives at the top of generate() in periods.py. Change it there, update test_writes_refused style tests to the new truth, and update the design note that recorded the old rule. Rules change in their owning layer plus their tests plus their paper trail, all three or it is not done.

Add a fifth service type

The two source of truth problem. SERVICE_TYPES plus SERVICE_LABELS in queries.py, AND the CHECK constraints frozen inside migration 001, which means a migration 002 rebuilding those tables (SQLite cannot edit constraints in place, new table, copy, drop, rename). Forget the migration and every insert of the new type dies on the old CHECK. This asymmetry is the sharpest edge in the codebase.

9. Drills, Answers Below the Line

Think first, write your answer down, then scroll.

- Drill 1, easy. Dad wants the client's phone shown on the client page. Which files change?
 - Drill 2, medium. Add GET /clients/{id}/history showing that client's tasks across ALL months. Which files, which new query, which existing function is the closest template to copy?
 - Drill 3, medium. New rule, a task in a CLOSED month may still move from detected to done. Where exactly does the current code refuse it, and what is the minimal correct change?
 - Drill 4, hard. The scanner arrives and must mark proofs detected. Which layer does the matching logic belong to, which existing columns receive the result, and which existing test file style do you copy for it?
 - Debug 1. Dashboard shows GSTR-3B pending 4, Priority's GSTR-3B section shows 3 rows. What do you check, in order?
 - Debug 2. Every form post suddenly returns 500. What one command do you run and which two causes from this project's own history do you suspect first?
-

Answers.

1. Nothing but client_detail.html, the phone column already travels through get_client. If it were a NEW column, see section 8.
 2. A route in clients.py, a new queries function joining tasks to periods filtered by client_id ordered by year and month, tasks_for_period is the shape to copy, plus a template and a test.
 3. set_task_status() in periods.py, the period status check. Minimal change, allow the transition when the task's proof_status is 'detected' and the target is 'done', refuse the rest, one test flips from refused to allowed, one new test guards the rest.
 4. Matching is logic, app/services/scanner.py, no HTTP in it. It writes proof_status 'detected' and proof_file_path, columns that have waited since migration 001. Tests copy the test_generation.py pattern, temp db, direct calls, no web.
 5. In order. Run the mirror test, if green the bug is in the PAGE not the queries, so compare the template loops, then check the URL params each page used, different period ids is the usual answer. If the mirror test is red, someone edited one query's WHERE.
 6. Run the app in a terminal and read the traceback of one post. The two historical suspects, python-multipart missing from the venv, and the SQLite cross thread error if someone bypassed connect().
-

10. Honest Review of This Codebase

Real findings from this audit, each with problem, why it matters, better approach, trade-off.

1. Duplicate generation logic. `seed.py` has its own `generate_tasks()`, written in v0.2 before the service existed. Two copies of the product's core statement can drift, a future change to one silently misses the other. Fix, `seed` imports and calls `generation.generate_tasks`, delete its local copy. Cost, one small PR. This is the first thing I would clean.
2. Duplicated period from URL parsing, `pick_period()` in `dashboard.py` versus an inline block in `clients.client_detail`. Same drift risk, smaller blast radius. Fix, move `pick_period` into `deps.py` or `queries` and call it from both. Cheap.
3. The service list lives in two worlds, Python constants and frozen schema CHECKs, section 8's sharpest edge. Acceptable at four services, document it in both places, pay the migration price when service five arrives.
4. error signalling via query params, `error=closed`. Stringly typed, invisible to type checkers, fine at this size, would become a proper flash message system in a bigger app. Leave it.
5. Inline styles and bare tables in templates. Deliberate, polish is a non goal until dad's trial proves the workflow. Leave it.
6. README staleness, section 1 above. Docs debt, worth a small PR.
7. Still true from the deep dive, no backup of `taxdesk.db`, the top pre trial item, and no CI running the 18 tests automatically.

What is genuinely good and should not be touched, the constraint driven schema, the single SQL boundary, idempotency everywhere, the mirror test, one directional imports, and connection lifecycle in exactly one place.

11. The Revision Sheets

Concept to code.

Concept	Where it lives	Why
routing	decorators in <code>app/routes/</code>	URL to function lookup
dependency injection	<code>Depends(get_db)</code> , <code>deps.py</code>	one correct connection lifecycle
parameterized SQL	every function in <code>queries.py</code>	injection impossible
JOIN	<code>pending_tasks</code> , <code>tasks_for_period</code>	ids become names
GROUP BY	<code>pending_counts</code> , <code>pending_clients</code>	rows become counts, computed not stored
UNIQUE as business rule	migration 001, <code>compliance_tasks</code>	duplicates impossible at the engine
idempotency	<code>generate_tasks</code> , <code>migrate()</code> , <code>seed</code>	repetition is harmless
transactions	<code>get_db</code> teardown, migrate loop	all or nothing per request, per migration

Concept	Where it lives	Why
post redirect get	every POST route, 303	refresh cannot double submit
trust boundary	confirm_clients, ALLOWED_STATUSES	input validated where it enters
fixtures	tests/conftest.py	fresh real schema per test
layering	the import graph itself	change one layer without the others

Code to concept, the reverse direction, pick a file and walk up.

```
migrate.py → transactions → migrations → schema evolution → reproducibility
queries.py → SQL → joins and aggregation → single source of truth
deps.py → resource lifecycle → dependency injection → statelessness per request
generation.py → set based SQL → idempotency → constraints as rules
periods.py → HTTP methods → PRG → business rule enforcement points
conftest.py → fixtures → test isolation → executable documentation
```

12. The Final Test, Answered

- a button is clicked, where does the request go. The routing table built at import time, method plus path, straight to one function in app/routes/
- data changes, which code changes it. Only functions in queries.py, called by POST routes, committed by get_db
- the database fails, where does the error go. The exception rises out of queries, through the route, FastAPI's error middleware turns an uncaught one into a 500, and get_db's teardown skips the commit so nothing partial persists
- I add a feature, which layers change. Section 8, usually migration plus queries plus route plus template plus test, and nothing else
- a business rule changes, where does it live. In its owning layer, route checks for web gated rules, services for logic, schema for invariants that must survive any bug
- the system slows, where do I look. The queries, with EXPLAIN QUERY PLAN, and the first index candidate is (period_id, status) on compliance_tasks
- a test fails, which code does it exercise. The table in section 5
- I open an unfamiliar file, can I tell why it exists. The journal's code map section answers in ten seconds

If you can answer all eight without this document open, you own the codebase.